



**St. Vincent's College Incorporated
Computer Engineering Department**



SMART TRASHBIN

**In Partial Fulfillment of the Requirements in Microprocessors for the
Computer Engineering Course, St. Vincent's College Incorporated,
Academic Year 2026-2027**

**Submitted by:
Nathaniel Joshua D. Yecpot
Jazz E. Asa
Angelo A. Taldo
January Anne S. Baidiango**

**Submitted to:
Mrs. Christine June M. Jumawan**

August 2026

Table of Contents

Acknowledgement	2
Introduction	3
Arduino Source Code	4
Circuit Schematic	14
Wiring Diagram	15
Block Diagram	15
Flow Chart	16
Bill of Materials	16
User Manual	17

Acknowledgement

We would like to express our heartfelt gratitude to our Microprocessors Instructor, Mrs. Christine June M. Jumawan, for her invaluable guidance, expertise, and unwavering support throughout the semester. Her dedication to her profession and commitment to nurturing her students' skills have been truly exemplary and instrumental in our growth as developers.

We also extend our sincere appreciation to our project partner for their collaboration, dedication, and contributions, which were essential in completing the Smart Trashbin Project.

Our deepest thanks go to our families and friends for their constant encouragement and support.

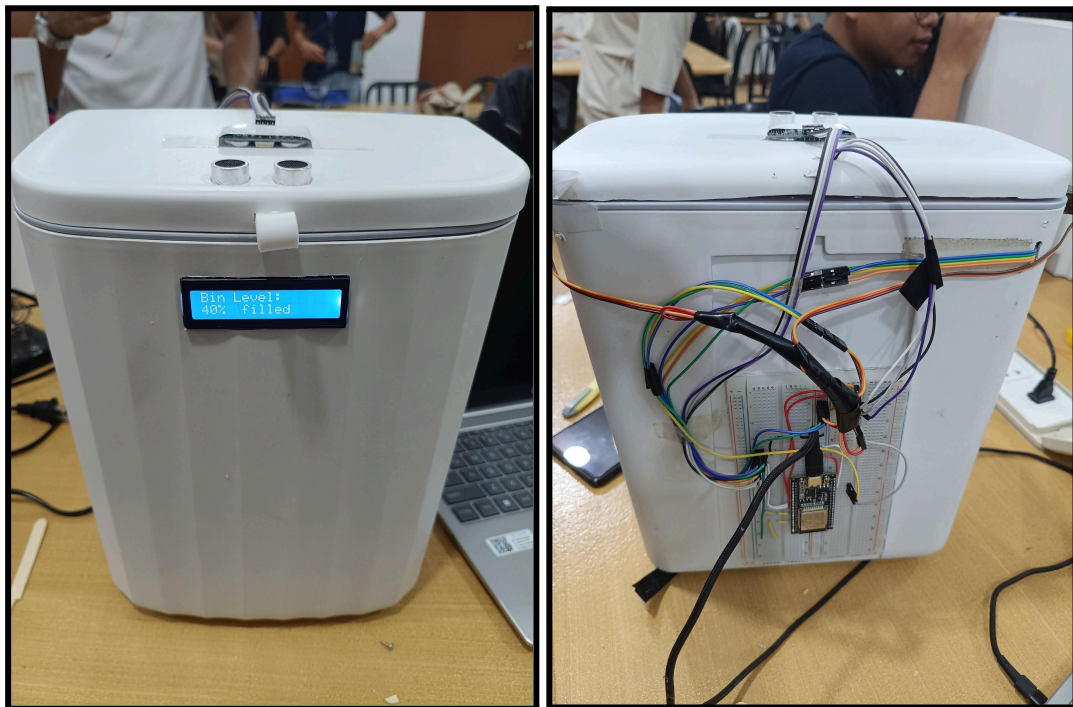
Lastly, we express our profound gratitude to God for His guidance, wisdom, and presence throughout the challenges and triumphs of this semester.

Introduction

The Smart Trashbin is a smart waste monitoring and automated waste bin system developed as a midterm project for the Microprocessors course in the Computer Engineering program. It is designed to monitor the amount of waste accumulated inside a trash bin using an ultrasonic sensor while also providing an automatic lid opening and closing mechanism for convenient, touch-free operation.

The system uses one ultrasonic sensor to continuously measure the distance between the sensor and the surface of the waste, allowing it to determine the current fill level of the trash bin. A second ultrasonic sensor is mounted on top of the lid and faces upward to detect when a user's hand is placed close to the sensor. When a hand is detected within the predefined distance, the ESP32 Dev Kit Module activates the servo motors to automatically open the lid. After a short delay, the lid automatically closes.

The measured data is transmitted by the ESP32 Dev Kit Module to a Supabase cloud database, where it is stored and made available to the web application. The web-based dashboard provides real-time information, including the trash bin's fill percentage, status, lid state, raw sensor distance, and the timestamp of the latest reading. It also sends notifications when the trash level reaches a predefined threshold, allowing users to monitor the bin remotely and reduce the need for frequent physical inspections.



Arduino Source Code

```
/*
=====
=====
SMART TRASH BIN — ESP32 firmware
Ultrasonic (fill level) + Ultrasonic (proximity/auto-open) + 2 Servos
(synced lid, separate pins) + 16x2 I2C LCD + Supabase upload

Libraries needed (install via Library Manager):
- LiquidCrystal I2C (by Frank de Brabander / Marco Schwartz)
- ArduinoJson (by Benoit Blanchon)
- ESP32Servo (by Kevin Harrington / madhephaestus)
(WiFi.h and HTTPClient.h ship with the ESP32 board package)

=====
===== */

#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ArduinoJson.h>
#include <ESP32Servo.h>
#include <time.h>

// ===== LABEL: FILL THESE IN
=====
const char* WIFI_SSID = "MERCUSYS_B8B1";
const char* WIFI_PASSWORD = "kisssalanggabi";

const char* SUPABASE_URL = "https://bxeaxpbmjgqvqcrbqzjql.supabase.co";
const char* SUPABASE_API_KEY=
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJkdXBhYmFzZSIsInJlZiI6ImJ4ZWZ4cGJtamd2cWNyYnF6anFsIiwicm9sZSI6ImFub24iLCJpYXQiOiE3ODU0OTc0MzAsImV4cCI6ImJEWMTA3MzQzMH0.qsEMpWHucU1oblZPqi_LtlthS26uoe2GPMVSQkvWYN8";
const char* SUPABASE_TABLE = "bin_data"; // table name from
supabase_schema.sql
//
=====
=====
```

```

// ===== LABEL: HARDWARE PINS
=====
// ---- Fill-level ultrasonic sensor (existing) ----
const int FILL_TRIG_PIN = 5;
const int FILL_ECHO_PIN = 18;

// ---- Proximity ultrasonic sensor (triggers auto lid open) ----
const int PROXIMITY_TRIG_PIN = 4;
const int PROXIMITY_ECHO_PIN = 19;

// ---- Lid servos (your actual wiring: two separate pins) ----
// NOTE: GPIO12 is a boot-strapping pin on the ESP32. If you ever see
// boot/reset flakiness, this pin is the first thing worth investigating
// (e.g. moving SERVO_2_PIN to a non-strapping pin like 14, 27, 26...).
// Left as-is here to match your actual current wiring.
const int SERVO_1_PIN = 13;
const int SERVO_2_PIN = 12;

// ---- LCD (I2C) ----
const uint8_t LCD_ADDRESS = 0x27; // common I2C addresses: 0x27 or 0x3F
const uint8_t LCD_COLS = 16;
const uint8_t LCD_ROWS = 2;
//
=====
=====

LiquidCrystal_I2C lcd(LCD_ADDRESS, LCD_COLS, LCD_ROWS);
Servo lidServo1;
Servo lidServo2;

// ---- Bin geometry ----
const float BIN_DEPTH_CM = 27.0; // distance from sensor to bin floor when
empty
const float EMPTY_THRESHOLD_CM = 26.0; // readings at/above this => treat as
0% (empty)

// ---- Sensor sanity bounds (HC-SR04 typical usable range) ----
const float SENSOR_MIN_CM = 2.0;
const float SENSOR_MAX_CM = 400.0;
const float JUMP_THRESHOLD_CM = 15.0;
const float JUMP_CONFIRM_TOLERANCE_CM = 3.0;
const int JUMP_CONFIRM_COUNT = 3;

```

```

// Distance beyond this, while the lid is closed, means a genuine sensor
// malfunction (not "lid open" — we already know the real lid state now
// from the servo, so we don't have to guess anymore).
const float LID_OPEN_MARGIN_CM = 10.0;
const float LID_OPEN_DISTANCE_CM = BIN_DEPTH_CM +
LID_OPEN_MARGIN_CM; // 37cm with current depth

float pendingDistance = -1;
int pendingConfirmCount = 0;

// ===== LABEL: AUTO-OPEN SETTINGS
=====
const float PROXIMITY_TRIGGER_DISTANCE_CM = 30.0;
const unsigned long PROXIMITY_CHECK_INTERVAL_MS = 200;
const unsigned long LID_AUTO_CLOSE_DELAY_MS = 8000;

// Servo angles. Both servos move the same physical lid together. If your
// two servos are mounted mirrored/opposite each other, set one of these
// pairs to (180 - angle) so they move in sync instead of fighting.
const int SERVO_1_OPEN_ANGLE = 124;
const int SERVO_1_CLOSED_ANGLE = 0;
const int SERVO_2_OPEN_ANGLE = 0;
const int SERVO_2_CLOSED_ANGLE = 120; // mirrored so servo 2 turns the
opposite way
//
=====
=====

bool lidsOpen = false;
bool previousLidOpenState = false; // tracks open->closed transitions
unsigned long lastProximityDetectedMs = 0;
unsigned long lastProximityCheckMs = 0;

const unsigned long READ_INTERVAL_MS = 3000;
const unsigned long UPLOAD_INTERVAL_MS = 5000;

float lastGoodDistance = -1;
unsigned long lastReadMs = 0;
unsigned long lastUploadMs = 0;

//
=====
=====
void setup() {

```

```

Serial.begin(115200);

pinMode(FILL_TRIG_PIN, OUTPUT);
pinMode(FILL_ECHO_PIN, INPUT);
pinMode(PROXIMITY_TRIG_PIN, OUTPUT);
pinMode(PROXIMITY_ECHO_PIN, INPUT);

lidServo1.attach(SERVO_1_PIN);
lidServo2.attach(SERVO_2_PIN);
closeLid(); // start in a known, closed position

lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Smart Bin");
lcd.setCursor(0, 1);
lcd.print("Connecting...");

connectWiFi();

configTime(0, 0, "pool.ntp.org", "time.nist.gov");
Serial.println("Waiting for NTP time sync...");
struct tm timeinfo;
while (!getLocalTime(&timeinfo)) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nTime synced.");
}

//
=====
=====
void loop() {
    unsigned long now = millis();

    if (now - lastProximityCheckMs >= PROXIMITY_CHECK_INTERVAL_MS) {
        lastProximityCheckMs = now;
        handleLidAutoOpen();
    }

    if (now - lastReadMs >= READ_INTERVAL_MS) {
        lastReadMs = now;
        processReading();
    }
}

```

```

    }
}

//
=====
=====
// AUTO-OPEN LOGIC (proximity sensor -> servos)
//
=====
=====
void handleLidAutoOpen() {
    float proximityDistance = readDistanceCm(PROXIMITY_TRIG_PIN,
PROXIMITY_ECHO_PIN);
    bool objectDetected = (proximityDistance > 0 && proximityDistance <=
PROXIMITY_TRIGGER_DISTANCE_CM);

    if (objectDetected) {
        lastProximityDetectedMs = millis();
        if (!lidIsOpen) openLid();
    } else if (lidIsOpen && (millis() - lastProximityDetectedMs >=
LID_AUTO_CLOSE_DELAY_MS)) {
        closeLid();
    }
}

void openLid() {
    lidServo1.write(SERVO_1_OPEN_ANGLE);
    lidServo2.write(SERVO_2_OPEN_ANGLE);
    lidIsOpen = true;
    Serial.println("Lid: OPEN (auto)");
}

void closeLid() {
    lidServo1.write(SERVO_1_CLOSED_ANGLE);
    lidServo2.write(SERVO_2_CLOSED_ANGLE);
    lidIsOpen = false;
    Serial.println("Lid: CLOSED (auto)");
}

//
=====
=====
// FILL-LEVEL LOGIC — driven by the REAL lid state from the servo

```

```
//
=====
=====
String classifyStatus(int percentage) {
  if (percentage <= 0)    return "empty";
  else if (percentage < 70) return "filled";
  else if (percentage < 90) return "critical";
  else                    return "full";
}

void processReading() {
  // Always take a raw reading for logging, even while the lid is open —
  // it just won't be trusted for the percentage calc in that case.
  float distance = readDistanceCm(FILL_TRIG_PIN, FILL_ECHO_PIN);

  bool lidJustClosed = (previousLidOpenState == true && lidIsOpen == false);
  previousLidOpenState = lidIsOpen;

  int percentage;
  String status;
  String binStatus = lidIsOpen ? "open" : "closed"; // ground truth from the servo

  if (lidIsOpen) {
    // Can't trust the fill sensor right now — it's looking at open space.
    status = "processing";
    percentage = lastGoodDistance > 0 ? distanceToPercentage(lastGoodDistance) :
0;
    pendingDistance = -1;
    pendingConfirmCount = 0;

  } else {
    bool tooClose = (distance > 0 && distance < SENSOR_MIN_CM);
    bool sensorError = (distance <= 0 || distance > SENSOR_MAX_CM || distance >
LID_OPEN_DISTANCE_CM);

    if (tooClose) {
      // Something touching the sensor = definitely full, accept immediately.
      lastGoodDistance = distance;
      percentage = 100;
      status = "full";
      pendingDistance = -1;
      pendingConfirmCount = 0;

    } else if (sensorError) {
```

```

    // We KNOW the lid is closed (servo confirms it), so this is a real
    // sensor malfunction, not "someone opened the lid".
    status = "processing";
    percentage = lastGoodDistance > 0 ? distanceToPercentage(lastGoodDistance) :
0;
    Serial.println("WARNING: fill sensor error while lid is closed — check
wiring/mounting.");

    } else if (lidJustClosed) {
        // Lid just came down — trash may have just been added during the
        // open period. Trust this reading immediately, no debounce needed.
        lastGoodDistance = distance;
        percentage = distanceToPercentage(distance);
        status = classifyStatus(percentage);
        pendingDistance = -1;
        pendingConfirmCount = 0;

    } else {
        // Lid has been closed a while — nothing should physically be able
        // to change the fill level right now. Treat a big jump cautiously.
        bool suddenJump = (lastGoodDistance > 0 && fabs(distance - lastGoodDistance)
>= JUMP_THRESHOLD_CM);

        if (!suddenJump) {
            pendingDistance = -1;
            pendingConfirmCount = 0;
            lastGoodDistance = distance;
            percentage = distanceToPercentage(distance);
            status = classifyStatus(percentage);
        } else {
            if (pendingDistance > 0 && fabs(distance - pendingDistance) <=
JUMP_CONFIRM_TOLERANCE_CM) {
                pendingConfirmCount++;
            } else {
                pendingDistance = distance;
                pendingConfirmCount = 1;
            }
        }

        if (pendingConfirmCount >= JUMP_CONFIRM_COUNT) {
            lastGoodDistance = distance;
            percentage = distanceToPercentage(distance);
            status = classifyStatus(percentage);
            pendingDistance = -1;
            pendingConfirmCount = 0;
        }
    }
}

```

```

    } else {
        percentage = distanceToPercentage(lastGoodDistance);
        status = classifyStatus(percentage);
    }
}
}
}

updateLCD(percentage, status);

Serial.printf("distance=%.1fcm percentage=%d%% status=%s bin=%s lid=%s\n",
    distance, percentage, status.c_str(), binStatus.c_str(),
    lidIsOpen ? "OPEN" : "CLOSED");

if (millis() - lastUploadMs >= UPLOAD_INTERVAL_MS) {
    lastUploadMs = millis();
    uploadToSupabase(percentage, status, binStatus, distance);
}

//
=====
=====
float readDistanceCm(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0) return -1;

    return duration * 0.0343 / 2.0;
}

int distanceToPercentage(float distanceCm) {
    if (distanceCm >= EMPTY_THRESHOLD_CM) return 0;
    float pct = ((BIN_DEPTH_CM - distanceCm) / BIN_DEPTH_CM) * 100.0;
    if (pct < 0) pct = 0;
    if (pct > 100) pct = 100;
    return (int) round(pct);
}

```

```
//
=====
=====
void updateLCD(int percentage, String status) {
    lcd.clear();

    if (status == "full") {
        lcd.setCursor(0, 0);
        lcd.print("!! BIN FULL !!");
        lcd.setCursor(0, 1);
        lcd.print("EMPTY IT NOW");
    } else if (status == "processing") {
        lcd.setCursor(0, 0);
        lcd.print(lidIsOpen ? "Lid Open..." : "Checking bin...");
        lcd.setCursor(0, 1);
        lcd.print(String(percentage) + "% (last)");
    } else {
        lcd.setCursor(0, 0);
        lcd.print("Bin Level:");
        lcd.setCursor(0, 1);
        lcd.print(String(percentage) + "% " + status);
    }
}

//
=====
=====
void connectWiFi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to WiFi: ");
    Serial.println(WIFI_SSID);

    unsigned long startAttempt = millis();
    const unsigned long WIFI_TIMEOUT_MS = 15000;

    while (WiFi.status() != WL_CONNECTED) {
        if (millis() - startAttempt > WIFI_TIMEOUT_MS) {
            Serial.println("\nWiFi connection FAILED.");
            Serial.print("Status code: ");
            Serial.println(WiFi.status());

            lcd.clear();
            lcd.setCursor(0, 0);

```

```

    lcd.print("WiFi Failed");
    lcd.setCursor(0, 1);
    lcd.print("Check creds/2.4GHz");

    delay(5000);
    startAttempt = millis();
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}
delay(500);
Serial.print(".");
}
Serial.println("\nWiFi connected. IP: " + WiFi.localIP().toString());
}

String getIsoTimestamp() {
    struct tm timeinfo;
    if (!getLocalTime(&timeinfo)) return "";
    char buf[30];
    strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S", &timeinfo);
    return String(buf);
}

//
=====
=====
void uploadToSupabase(int percentage, String status, String binStatus, float
distance) {
    if (WiFi.status() != WL_CONNECTED) {
        connectWiFi();
    }

    HTTPClient http;
    String endpoint = String(SUPABASE_URL) + "/rest/v1/" + SUPABASE_TABLE;

    http.begin(endpoint);
    http.addHeader("Content-Type", "application/json");
    http.addHeader("apikey", SUPABASE_API_KEY);
    http.addHeader("Authorization", String("Bearer ") + SUPABASE_API_KEY);
    http.addHeader("Prefer", "return=minimal");

    StaticJsonDocument<256> doc;
    doc["percentage"] = percentage;
    doc["status"] = status;

```

```

doc["bin_status"] = binStatus;
doc["distance_cm"] = distance;

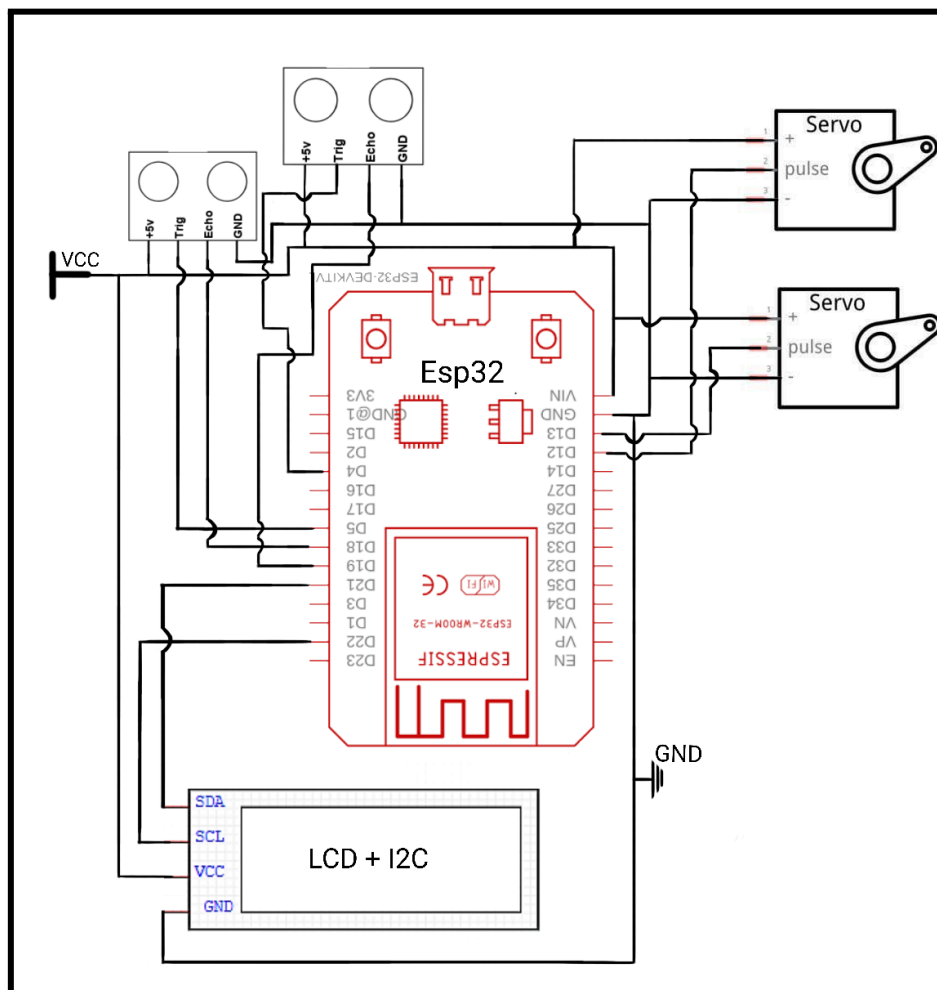
String isoTime = getIsoTimestamp();
if (isoTime.length() > 0) {
  doc["recorded_at"] = isoTime;
}

String payload;
serializeJson(doc, payload);

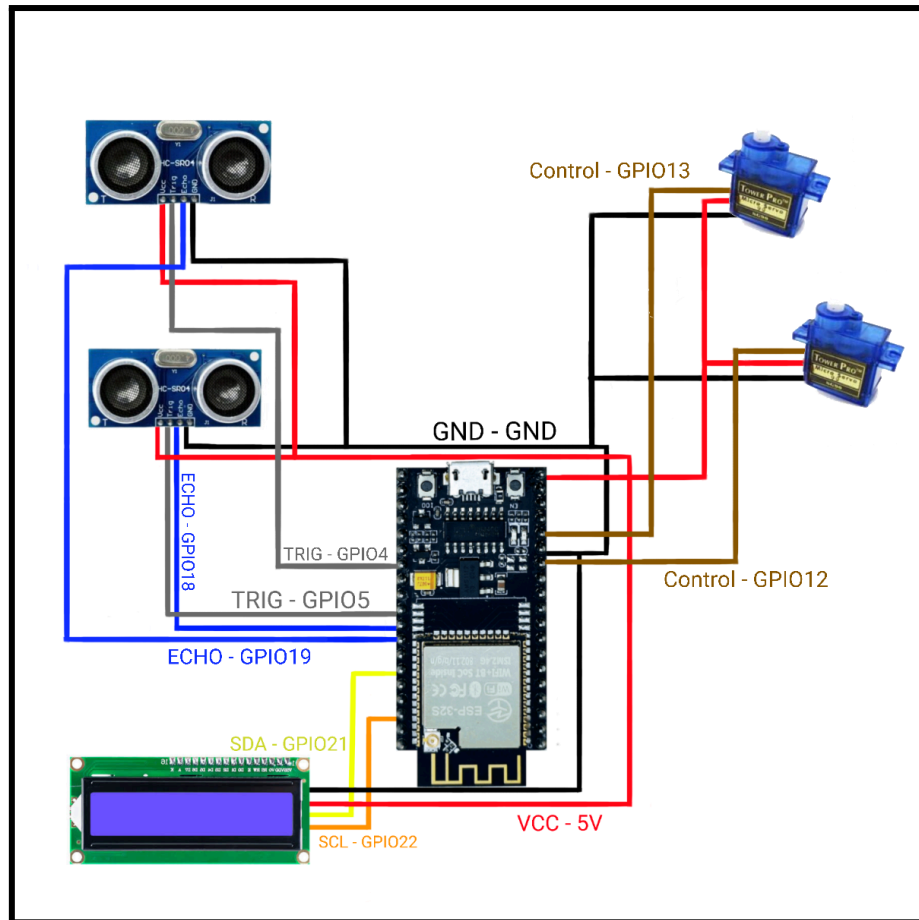
int responseCode = http.POST(payload);
Serial.printf("Supabase upload response: %d\n", responseCode);
if (responseCode != 200 && responseCode != 201) {
  Serial.println("Response body: " + http.getString());
}
http.end();
}

```

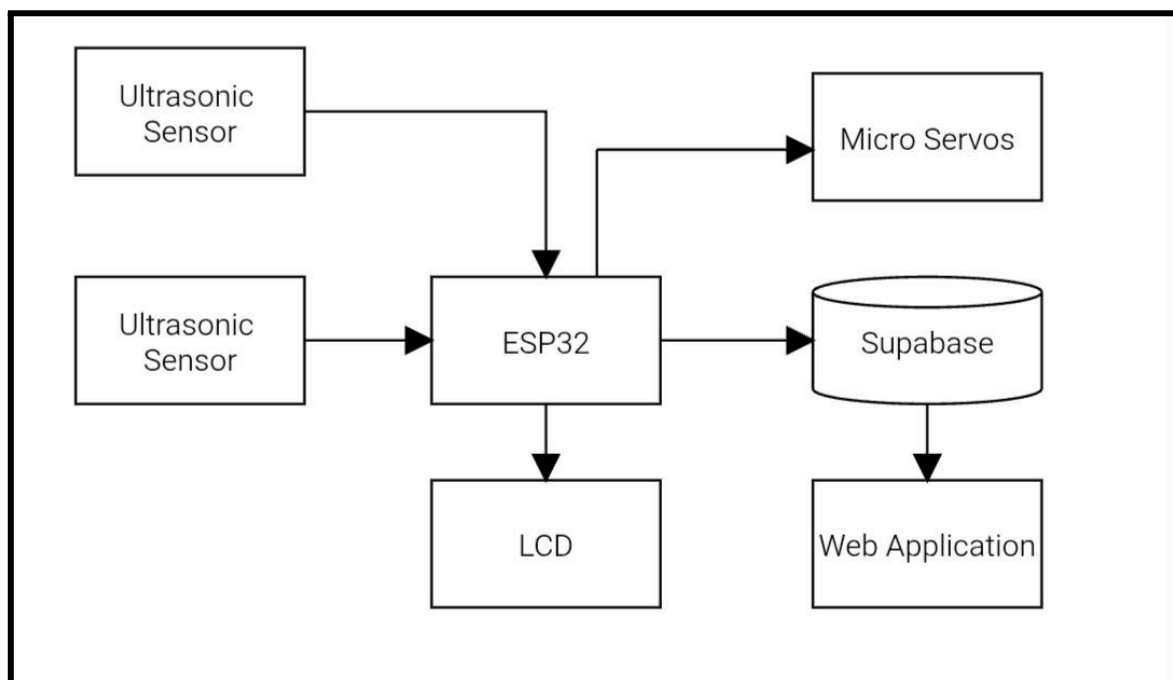
Circuit Schematic



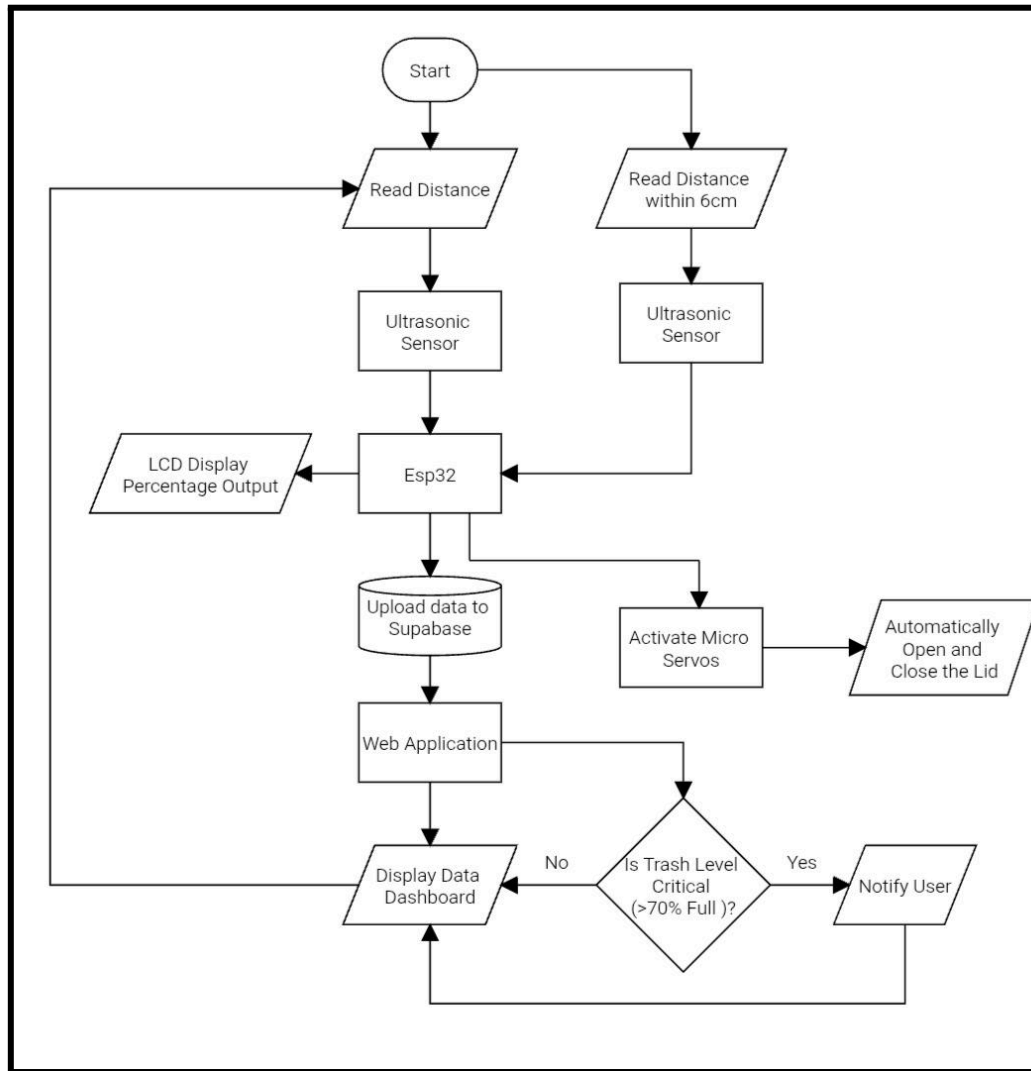
Wiring Diagram



Block Diagram



Flow Chart



Bills of Materials

Items	Quantity	Unit Cost	Amount
LCD with I2C	1	165.00	165.00
Ultrasonic Sensor	2	65.00	130.00
ESP32	1	265.00	265.00
Breadboard	2	80.00	160.00
Wires	22	10.00	220.00
Trashbin	1	289.00	289.00
Micro Servo	2	90.00	180.00
TOTAL AMOUNT			1,409.00

User Manual

1. Connect a 5 V USB Type-C power adapter to the ESP32 Dev Kit Module to power on the smart trash bin.

2. Open a web browser and visit: <https://trickscorp.github.io/Smart-Trashbin/>

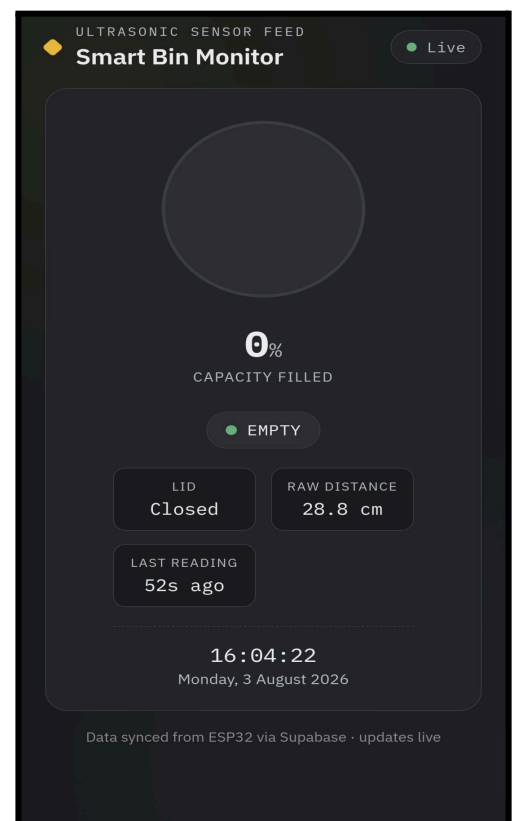
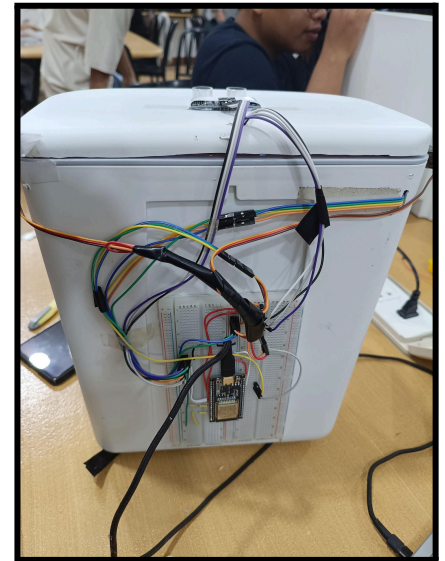
You may also install the web application on your device for easier access.

3. Once the dashboard loads, the following information will be displayed:

- Capacity Filled – Displays the percentage of the trash bin that is occupied by waste.
- Status – Indicates the current fill level of the trash bin:
 - Empty: 0%
 - Filled Up: 1–69%
 - Critical: 70–89%
 - Full: 90–100%
 - Checking Sensor: Displays when sensor reading is not stabilised.
- Lid – Indicates whether the trash bin lid is Open or Closed.
- Raw Distance – Displays the distance measured by the ultrasonic sensor.
- Last Reading – Displays the time of the most recent sensor reading uploaded to the database.

4. The web application automatically notifies the user when the trash level reaches or exceeds the predefined threshold.

5. To use the Smart Trashbin's automatic open-close system, simply place your hand close to the upward-facing ultrasonic sensor mounted on top of the lid. The lid will automatically open when your hand is detected and will close automatically after a 5-seconds delay.



Note: If the raw distance readings show abnormal behavior, please inspect the trash bin to ensure it is in proper operating condition.